École Nationale Supérieure Polytechnique de Yaoundé

Université de Yaoundé I

PROJET ÉVALUÉ — APPLICATION DÉPLOYÉE SUR AWS

Rapport de test

Méthode, objectifs, résultats bruts, analyse et preuve des deux exigences éliminatoires

Éditeur de documents partagé pour le journal d'une association étudiante

26 octobre 2026

Sommaire

0. Objectifs fixés

1. Méthode générale

2. Nombre exact de téléchargements — jamais un de plus

3. Aucune modification perdue — et toutes les copies convergent

4. PDF à jour au moment du clic

5. Crédit quotidien d’écriture — jamais plus, même depuis plusieurs appareils

6. Tenue du pic d’affluence et annonce rapide de l’épuisement

7. Autres fonctions testées (Vitest)

8. Conclusion

Méthode, objectifs, résultats bruts, analyse, et preuves des deux exigences éliminatoires du sujet : le **nombre exact de téléchargements** et **aucune modification perdue**. Chaque mesure écrit ses **résultats bruts horodatés** dans `tests/results/` (JSON, réutilisables tels quels) ; ce rapport en cite les chiffres et donne le fichier. Commandes pour tout rejouer : README.

0. Objectifs fixés

Exigence du sujet	Objectif mesurable	Seuil de réussite	Où
Nombre exact de téléchargements (éliminatoire)	N ≫ K clics simultanés sur un lien de K exemplaires	exactement K réponses 200/202 avec réservation, tout le reste 410 ; en base `remaining = 0` et K réservations ; sur plusieurs tirages, dont K = 1 contre 2 000 clics	§2
Aucune modification perdue (éliminatoire)	≥ 20 clients Yjs écrivent **au même endroit** en même temps, y compris pendant un arrêt brutal du serveur temps réel	textes et vecteurs d'état identiques ; état relu depuis la base identique ; 0 séquence confirmée manquante ; longueur finale = frappes envoyées	§3
Aucune modification perdue (mesure complémentaire)	retard de confirmation dans le temps, à 8 caractères/s par client	retard stable (sinon : regroupement côté client)	§3
PDF à jour au moment du clic	un lien créé avant une modification confirmée, cliqué après	PDF de séquence ≥ dernière séquence confirmée avant le clic, PDF complet	§4
Crédit quotidien d'écriture	un même e-mail sur 3 appareils et 2 documents, en rafale	total accepté = crédit débité ≤ 10 000	§5
Tenue du pic d'affluence	15 000 visiteurs (80 % en 1 min) pendant que 300 rédacteurs écrivent	0 visiteur sans réponse finale, 0 % d'échec, exactement le stock servi	§6
Annonce rapide de l'épuisement	réponse « épuisé » après le dernier exemplaire	410 rapide, sans accès à DynamoDB ; le client cesse d'interroger	§6

1. Méthode générale

Niveau	Outil	Contre quoi	Quand
Unitaires et intégration	Vitest 4 (115 tests, `npm test`)	DynamoDB Local, vrais serveurs Fastify et Hocuspocus en mémoire, vrais clients Yjs	À chaque étape

Niveau	Outil	Contre quoi	Quand
Infrastructure	Vitest + `aws-cdk-lib/assertions`	Gabarits CloudFormation synthétisés (IAM exacte, aucun `Resource: *` hors exception, ports, étiquettes)	À chaque étape
Scénarios de contraintes	Scripts `tests/c1` à `tests/c4` (un par exigence, dans l'ordre du sujet)	Local **et** déploiement AWS (CloudFront → ALB → instances)	À chaque fonction livrée
Charge (pic d'affluence, annonce de l'épuisement)	k6 v2.2.0 + 300 vrais clients Yjs	Déploiement AWS, depuis une instance EC2 dédiée en eu-north-1 (`EditeurPartageCharge`)	Une fois l'application complète

Les clients de test des scénarios temps réel sont de vrais `HocuspocusProvider` + Yjs, comme le navigateur. Les vérifications « en base » relisent DynamoDB et S3 en lecture seule.

2. Nombre exact de téléchargements — jamais un de plus

Objectif. K exemplaires, N ≫ K clics simultanés (≥ 1 000) : exactement K réponses 200/202 avec réservation, tout le reste 410 ; en base, `remaining = 0` et exactement K réservations. Plusieurs tirages.

Méthode (`tests/c1/run-c1.ts`). Un document et un lien de K exemplaires par tirage ; N clics lancés **en même temps** (HTTP/2 multiplexé, une connexion, pour que les requêtes arrivent ensemble et non à la file) ; chaque visiteur est un client sans cookie. En local, 3 instances d'API **à mémoire séparée** se partagent les clics (la mémoire « épuisé » d'une instance ne doit rien changer). Puis rejeu : chaque gagnant revient avec son ticket et doit retrouver **sa** réservation, sans nouveau ticket. Enfin, relecture de la base.

Résultats bruts.

Fichier	Cible	K	N	Tirages	200/202	410	`remaining`	Réservations	Verdict
`c1/2026-10-25T11-59-18-663Z-local.json`	local, 3 instances	500	1 200	3	500 · 500 · 500	700 · 700 · 700	0 · 0 · 0	500 · 500 · 500	OK
`c1/2026-10-25T11-59-48-004Z-local.json`	local, 3 instances	1	2 000	3	1 · 1 · 1	1 999 × 3	0 × 3	1 × 3	OK

Fichier	Cible	K	N	Tirages	200/202	410	`remaining`	Réservations	Verdict
`c1/2026-10-25T16-46-50-815Z-aws.json`	AWS	500	1 200	3	500 · 500 · 500	700 · 700 · 700	0 × 3	500 × 3	OK
`load/2026-10-25T23-02-10Z/` (test de charge)	AWS, 4 instances	5 000	15 819 visiteurs	1	5 000	10 819	0	5 000	OK

Le rejeu des gagnants a renvoyé leur réservation dans 100 % des cas, sans aucun nouveau ticket. Erreurs réseau : 0.

Tirages écartés, et pourquoi.

- `c1/2026-10-25T16-42-41-290Z-aws.json` : 1 199 erreurs réseau (statut 0) — premier essai sur AWS, le client ouvrait 1 200 connexions TLS d'un coup depuis un poste personnel. Corrigé en passant le client à HTTP/2 multiplexé ; **aucun exemplaire en trop** (1 seul servi).
- `c1/2026-10-26T00-15-44-573Z-aws.json` : 1 470 et 1 500 réponses **429** — tirage lancé après la mise en place de l'anti-abus depuis un seul poste : au-delà de 30/N requêtes en 10 s, l'adresse est bloquée, comme prévu. La base confirme qu'aucun exemplaire n'a été donné en trop (1 réservation pour K = 1 au premier tirage ; 0 au second, `remaining = 1`). Depuis l'anti-abus, la preuve à grande échelle sur AWS est le test de charge (adresse exemptée) ; `test:c1` sur AWS se lance depuis l'instance de test.

Analyse. Le décrément `remaining = remaining − 1` conditionné à `remaining > 0` est exécuté par DynamoDB élément par élément : la concurrence ne peut pas produire plus de K succès. Les tirages à K = 1 contre 2 000 clics sont le cas le plus dur (un seul gagnant possible) ; le test de charge confirme à l'échelle du pic (5 000 exemplaires, ≈ 200 clics/s, 4 instances). **L'exigence est tenue dans tous les tirages exploitables : jamais un exemplaire de plus, exactement K quand la demande dépasse l'offre.**

3. Aucune modification perdue — et toutes les copies convergent

Objectif. ≥ 20 clients Yjs qui écrivent **au même endroit** en même temps ; états finaux identiques (texte **et** vecteur d'état) ; chaque `seq` confirmé présent dans le document final ; état relu depuis la base identique ; mesure du **retard de confirmation dans le temps** à 8 caractères/s par client.

Méthode (`tests/c2/run-c2.ts`, vérifications communes dans `tests/c2/convergence.ts`). 20 rédacteurs insèrent chacun 8 caractères/s **en position 0** (même endroit) pendant 60 s, une frappe = un message (le pire cas). Retard de confirmation = délai entre la frappe et l'accusé `ack(seq)` du serveur, envoyé **après** l'écriture dans DynamoDB, regroupé par fenêtres de 5 s. Après la frappe : attente des confirmations, puis de la diffusion des dernières modifications à toutes les copies, puis :

- textes et vecteurs d'état des 20 clients identiques ;
- état rechargé depuis DynamoDB + S3 (instantané + journal) identique aux clients ;

- pour **chaque** `seq` confirmé à un client : l'élément de journal existe, et son update est contenu dans l'état final (`Y.snapshotContainsUpdate`).

Résultats bruts.

Fichier	Cible	Frappes	Confirmées	Refus	Retard p50 / p95 (début → fin)	Convergence	Verdict
`c2/2026-10-25T18-51-20-324Z-local.json`	local, 60 s	9 521	9 521	0	98/144 → 90/139 ms	textes, vecteurs, base identiques ; 596 seq confirmés, 0 manquant	OK
`c2/2026-10-25T18-50-42-019Z-local.json`	local, 30 s	4 746	4 746	0	p95 139 → 138 ms	identiques ; 298 seq, 0 manquant	OK
`c2/2026-10-26T00-11-27-013Z-aws-attente-diffusion.json`	AWS, 60 s	9 552	9 552	0	1 113/1 929 → 913/1 520 ms (stable)	identiques ; 579 seq, 0 manquant ; base identique	OK
`c2/2026-10-26T00-30-51-335Z-local.json`	local, 20 s, regroupement 200 ms	3 180	3 180	0	p95 363 → 338 ms	identiques ; 159 seq, 0 manquant	OK
`c2/2026-10-26T00-35-17-613Z-aws.json`	AWS (après la refonte de l'éditeur), 60 s	9 547	9 547	0	p95 1 810 → 1 854 ms (stable)	identiques ; 562 seq, 0 manquant ; base identique	OK
`c2/2026-10-26T00-37-16-279Z-aws.json`	AWS, 60 s, regroupement 200 ms	9 560	9 560	0	p95 1 045 → 1 305 ms (stable)	identiques ; 473 seq, 0 manquant ; base identique	OK
`c2/2026-10-26T00-46-41-544Z-aws.json`	AWS, front redéployé, regroupement 200 ms	9 541	9 541	0	p95 978 → 1 253 ms (stable)	identiques, 0 manquant ; base identique	OK

Fichier	Cible	Frappes	Confirmées	Refus	Retard p50 / p95 (début → fin)	Convergence	Verdict
`c2/2026-10-26T03-55-23-253Z-aws.json`	AWS, 10 s, liaison du poste dégradée	1 580	1 580	0	p95 12,9 → 22,8 s (croissant [1])	identiques ; 116 seq, 0 manquant ; base identique	OK [1]
`c2/2026-10-26T03-59-25-925Z-aws.json`	AWS, 180 s, regroupement 200 ms, **arrêt brutal à 04:01:01 UTC**	26 354	25 091 + 1 263 renvoyées [2]	0	p95 1,9 → 13,6 s (coupure [3])	longueur 26 354 = envoyées ; identiques ; 1 222 seq, 0 manquant ; base identique	**OK**

[1] Le fichier porte `ok: false` : le script comptait alors la tendance du retard dans son verdict. Côté serveur, au même moment, les journaux « mesure confirmation » donnaient p95 111 ms et 1,6 % de CPU ; la page `/sante` mettait 8 s à répondre au poste (ping 320 à 470 ms). Le retard croissant venait de la liaison du poste, pas du serveur. Depuis, le script sépare les deux : le verdict ne porte que sur les pertes et la convergence, et un retard croissant est affiché comme une alerte (`latencyTrend.growing` dans le JSON).

[2] Frappes en vol au moment de la coupure : renvoyées par la resynchronisation à la reconnexion, donc sans accusé rattaché à la frappe, mais toutes présentes dans le document final et en base.

[3] La coupure et la reconnexion (56 déconnexions sur 20 clients) font monter le retard ; la liaison du poste était dégradée pendant ce tirage.

Tirages écartés, et pourquoi.

- `c2/2026-10-25T18-49-40-901Z-local.json` : retard croissant (p95 775 → 2 278 ms), premier essai avec une fenêtre fixe de 300 ms par document. Cause : Hocuspocus traite les messages d'une connexion un par un, chacun attendant sa transaction ; une fenêtre fixe limitait chaque rédacteur à ~3 frappes/s. Corrigé par la boucle « une transaction dès que la précédente est finie, ≥ 100 ms entre deux » (dossier §5.3) : les deux tirages suivants sont plats. Convergence déjà parfaite dans ce tirage (rien de perdu, seulement lent).
- `c2/2026-10-26T00-08-23-584Z-aws.json` : « divergence » d'un client (9 541 caractères au lieu de 9 550). Cause : **le test comparait trop tôt** — l'accusé de réception d'un client part un instant avant la diffusion de sa modification aux autres copies. Rejoué avec attente de la diffusion (2,2 s ont suffi) : textes, vecteurs et base identiques. La vérification attend désormais la diffusion (60 s au plus) avant de comparer ; une vraie divergence resterait visible à l'expiration.

Retard de confirmation. En local, plat autour de 90 ms (médiane). Sur AWS, **stable mais élevé** : médiane 0,4 à 1,1 s, p95 0,5 à 1,9 s, sans tendance à la hausse sur 60 s. Cause : à 8 frappes/s et un message par frappe, les messages d'un rédacteur font la queue derrière les transactions DynamoDB de son document (un message à la fois par connexion). Remède prévu dès la conception si le retard se dégradait : **regroupement côté client**. L'éditeur envoie désormais ses frappes groupées toutes les 200 ms (`flushDelay` du provider Hocuspocus) ; le script mesure les deux cas (`--flush-delay 0` : pire cas ; `200` : éditeur réel). Sur AWS, le regroupement réduit le p95 d'environ 40 % (≈ 1,8 s → 1,0 à 1,3 s) et le nombre de transactions de 16 % (473 au lieu de 562 pour un nombre de frappes équivalent).

Part du réseau dans ces chiffres. Les mesures d'écriture sur AWS sont lancées depuis un poste personnel au Cameroun, vers Stockholm via CloudFront : chaque confirmation inclut cet aller-retour et les files d'attente qu'il crée. Depuis l'instance de test placée dans la région (test de charge, §6), les 300 rédacteurs avaient une médiane de 85 ms et un p95 d'environ 150 ms en début de test. Le retard propre au serveur est mesuré par le serveur lui-même et écrit dans ses journaux (`mesure confirmation`, §6).

Arrêt brutal de l'instance temps réel (`c2/2026-10-26T03-59-25-925Z-aws.json`, provoqué volontairement). 20 clients × 8 caractères/s pendant 180 s, au même endroit. À 04:01:01 UTC, en pleine frappe, le noyau de l'instance `i-0005024579b402fcd` est redémarré de force (sysrq) : Node est tué net, sans arrêt propre ni vidage. Le serveur est reparti seul (journal « serveur temps réel démarré », document rechargé depuis la base à `seq` 602). Résultat : 26 354 frappes envoyées, document final de 26 354 caractères ; 20 copies, vecteurs d'état et état relu depuis la base identiques ; les 1 222 séquences confirmées sont toutes présentes. **Aucune modification acceptée n'a été perdue.** C'est ce que garantit la construction : une modification n'est confirmée qu'après son écriture dans DynamoDB ; au redémarrage, le serveur recharge instantané + journal ; les clients renvoient ce qui n'était pas confirmé et Yjs ignore les doublons.

Analyse. Dans tous les tirages exploitables, local et AWS : **toutes** les frappes envoyées ont été confirmées, **toutes** les séquences confirmées sont dans le journal et dans l'état final, et les 20 copies ainsi que l'état relu depuis la base sont identiques au caractère et au vecteur d'état près, alors que les 20 clients écrivaient au même endroit. **L'exigence est tenue.**

4. PDF à jour au moment du clic

Méthode (`tests/c3/run-c3.ts` sur AWS ; `tests/c3/c3.test.ts` en local). Un rédacteur écrit, un lien est créé (PDF préparé), une modification est **confirmée**, puis un nouveau visiteur clique sur le **même** lien (créé avant la modification) pendant qu'une autre modification part juste après le clic. On télécharge le PDF et on vérifie que sa séquence (`pdfSeq`, inscrite dans le PDF) est ≥ la dernière séquence confirmée avant le clic. En local, le test vérifie aussi que le texte rendu est **exactement** l'état du journal à cette séquence (aucun état intermédiaire incohérent).

Fichier	Préparation (clic sans modification)	Clic après modification	Verdict
`c3/2026-10-25T19-48-02-778Z-aws.json`	200 en 1,7 s	200 sans rappel, `pdfSeq` 2 ≥ dernier ack 2, PDF valide	OK
`c3/2026-10-26T00-08-58-684Z-aws.json`	200 en 0,3 s	200 sans rappel, `pdfSeq` 2 ≥ dernier ack 2, PDF valide	OK
`c3/2026-10-26T00-35-08-736Z-aws.json` (après la refonte de l'éditeur : un seul lien, créé avant la modification)	—	202 puis 200 après 1 rappel, `pdfSeq` 3 ≥ dernier ack 2, PDF valide	OK
`c3/2026-10-26T00-46-01-168Z-aws.json` (front redéployé)	200 en 0,5 s	202 puis 200 après 1 rappel, `pdfSeq` 3 ≥ dernier ack 2, PDF valide	OK
`c3/2026-10-26T04-20-32-642Z-aws.json` (après le déploiement du 26/10 04:13)	200 en 1,5 s	202 puis 200 après 1 rappel (4,2 s), `pdfSeq` 3 ≥ dernier ack 2, PDF valide	OK

5. Crédit quotidien d’écriture — jamais plus, même depuis plusieurs appareils

Méthode. En local (Vitest, `tests/c2/c2.test.ts`) : un même e-mail sur deux connexions écrit en rafale jusqu’au refus ; total accepté ≤ 10 000. Sur AWS (`tests/c4/run-c4.ts`) : un même e-mail sur **3 appareils** (2 sur un document A, 1 sur un document B) envoie des blocs de 700 caractères toutes les 50 ms sans attendre les confirmations ; on relit les deux documents depuis la base et le crédit consommé.

Fichier	Envoyés	Acceptés (A + B)	Crédit en base	Refus	Verdict
`c4/2026-10-26T00-17-29-641Z-aws.json`	35 700	6 300 + 3 500 = 9 800	9 800	4001 × 3	OK
`c4/2026-10-26T00-34-48-128Z-aws.json` (après la refonte de l’éditeur)	38 500	9 800	9 800	4001 × 3	OK
`c4/2026-10-26T00-46-11-341Z-aws.json` (front redéployé)	—	9 800	9 800	4001 × 3	OK
`c4/2026-10-26T04-20-51-194Z-aws.json` (après le déploiement du 26/10 04:13)	35 000	6 300 + 3 500 = 9 800	9 800	4001 × 3	OK

9 800 = 14 blocs de 700 : le 15e aurait dépassé 10 000. Le total accepté est égal au crédit débité (le débit est dans la même transaction que le journal) et ne dépasse jamais 10 000, malgré trois appareils sur deux documents. Les trois connexions ont reçu le refus 4001 et sont passées en lecture seule.

6. Tenue du pic d’affluence et annonce rapide de l’épuisement

Objectif. 15 000 visiteurs, 80 % dans la première minute (≈ 200 requêtes/s), pendant que 300 rédacteurs écrivent ; chaque visiteur reçoit une réponse finale (télécharge / épuisé) ; après le dernier exemplaire, les visiteurs l’apprennent vite, sans solliciter les services.

Méthode (`tests/load/run.sh`, sur une instance c7i-flex.large en eu-north-1, créée pour le test puis détruite).

1. Un document, et un lien de **5 000 exemplaires** qui ouvre 12 minutes plus tard : la création programme la montée de l’ASG API (4 instances à `opensAt − 10 min`).
2. **300 rédacteurs** (vrais clients Yjs) sur 30 documents, 2 caractères/s chacun, avant, pendant et après le pic (18 min 48 s).
3. À l’ouverture, **k6** : 200 nouveaux visiteurs/s pendant 60 s, puis 12/s pendant 4 min. Chaque visiteur a son propre pot de cookies (ticket), suit `Retry-After` (202/425) et s’arrête au premier 200 ou 410.

Résultats bruts (`tests/results/load/2026-10-25T23-02-10Z/`).

Mesure	Résultat
Visiteurs	15 819
Ont téléchargé (200)	**5 000**
« Épuisé » (410)	**10 819**
Sans réponse finale	**0**
Exemplaires réservés	5 000 = stock (base : `remaining = 0`, 5 000 réservations, lien `epuise`)
Requêtes HTTP / échecs	15 892 / **0,00 %**
Durée des requêtes p50 / p95 / max	9 ms / 41 ms / 660 ms
Temps jusqu'à la réponse finale p95	41 ms
410 p95	**10 ms** (servi depuis la mémoire de l'instance)
Rappels (202)	73 au total
Instances API au pic	4 (montée programmée exécutée avant l'ouverture)
Rédacteurs : frappes / confirmées / refus	578 762 / 578 762 / 0
Rédacteurs : documents convergents	**30 / 30**
CPU du serveur temps réel (moyennes 5 min)	17 → 41 → 42 → 38 % (2 vCPU)

Analyse — pic d'affluence. Chaque visiteur a reçu une réponse finale, sans aucun échec ; 5 000 téléchargements exactement (le nombre exact de téléchargements, vérifié à l'échelle du pic). Le PDF étant préparé avant l'ouverture, les gagnants ont été servis en 200 directement (73 rappels seulement sur 5 000).

Analyse — annonce de l'épuisement. Après épuisement, un 410 coûte 10 ms au 95e centile et **aucun** accès à DynamoDB (mémoire « épuisé » de chaque instance) ; le client arrête d'interroger dès le premier 410.

Les rédacteurs pendant le pic, et une fausse alerte. Toutes leurs frappes ont été confirmées et les 30 documents ont convergé. Le retard de confirmation mesuré a d'abord semblé croître (p50 85 → 220 ms, p95 150 → 630 ms au fil des 19 minutes). L'analyse a montré que c'était le **générateur** qui saturait, pas le serveur :

- le générateur a envoyé 578 762 frappes au lieu des 676 800 prévues (−15 %) : sa boucle d'événements était pleine ;
- la hausse commence avant l'ouverture du lien (sans lien avec le pic) et le CPU du serveur est resté stable (≈ 40 %, sans bridage : instance en mode *unlimited*) ;
- reproduit en local : générateur à 100 % de boucle d'événements, serveur entre 25 et 47 %. Profil CPU : 29 % du temps du générateur dans la recherche de position de Yjs (`findMarker`), parce que les 300 clients simulés inséraient à des positions aléatoires dans des documents de plus en plus longs, sur un seul processus Node.

Le générateur a été corrigé : répartition sur plusieurs processus (un par cœur, chaque document entier dans un processus), curseur propre à chaque rédacteur (comme un humain), mesure de sa propre charge par fenêtre (une fenêtre où il dépasse 90 % est signalée comme non représentative du serveur). Rejoué en local au **double** du débit du test AWS (300 rédacteurs à 4 caractères/s, 6 min) : retard serveur plat, médiane 61 à 70 ms, p95 107 à 178 ms, boucle d'événements du serveur stable (70 à 77 %), générateur à 28 à 61 %, toutes les frappes confirmées, 30/30 documents convergents. Le serveur temps réel écrit aussi désormais sa propre mesure toutes

les 30 s dans ses journaux CloudWatch (`mesure confirmation` : nombre, p50, p95, max, charge de la boucle d'événements), indépendante de tout générateur. Un nouveau passage du test de charge sur AWS avec le générateur corrigé reste à faire pour publier la courbe serveur.

Ce que le test n'a pas exercé. Les 15 000 visiteurs partent d'une seule machine, dont l'adresse est exemptée de l'anti-abus pendant le test : le cas de visiteurs légitimes derrière un même NAT (campus, 4G) n'est pas mesuré (voir dossier §8, limite assumée).

7. Autres fonctions testées (Vitest)

Domaine	Ce qui est vérifié
Refus et resynchronisation	4001 à exactement 10 000 caractères ; Y.Doc détruit et recréé ; contenu non texte ou format inconnu refusé (4400) ; ni le format ni la suppression ne débitent de crédit
Anti-abus rédacteur	5 refus en 1 min → 4003 pendant 5 h ; le rédacteur bloqué lit toujours le document
Anti-abus visiteur	seuil 30/N, blocage 10 min écrit en base et partagé, 429 + `Retry-After`, adresse exemptée
Liens	un seul lien actif par document (deux créations simultanées → une seule), fermeture (410 ensuite ; réservation en attente honorée, exemplaire déjà remis non retéléchargeable ; actions programmées retirées), compteur affiché décroissant, renommage, droits
URL expirée	nouvelle URL sur le même PDF, une seule valide à la fois, même sous retours simultanés
PDF	un seul rendu pour des demandes simultanées, `pdfSeq` ne recule jamais, document fermé rechargé, mise en forme rendue avec les bonnes polices
Versions	version nommée, aperçu, restauration (texte et mise en forme) comme nouvelle modification sans crédit, refus d'un instantané d'un autre document
Admin	jeton à temps constant, documents ouverts, liens actifs, blocages filtrés sur `expireLe`
Vérifié sur AWS le 26/10 (après le déploiement de 04:13)	e-mail du dernier exemplaire : lien de 1 exemplaire, clic 200 puis 410, journal « dernier exemplaire réservé », SES `SentLast24Hours` = 1 (un seul envoi) ; fermeture d'un lien à heure d'ouverture : la montée programmée disparaît, la descente est avancée (minimum 1)
Infrastructure	IAM exacte par rôle, seule exception `DescribeAutoScalingGroups` , ALB réservé à CloudFront, `/internal*` bloqué, mise à jour temps réel sans chevauchement, étiquettes

8. Conclusion

Exigence du sujet	Preuve	État
Nombre exact de téléchargements	3 tirages locaux à K = 500, 3 à K = 1 contre 2 000 clics, 3 tirages AWS à K = 500, et 5 000 / 5 000 au pic : jamais un de plus	**Tenue**
Aucune modification perdue	20 clients au même endroit, local et AWS : 100 % confirmé, 0 seq manquant, copies et base identiques	**Tenue**, y compris après arrêt brutal du serveur temps réel en pleine frappe
PDF à jour au moment du clic	PDF à `pdfSeq ≥` dernière modification confirmée avant le clic, sur AWS ; texte exact en local	Tenue
Crédit quotidien d'écriture	3 appareils, 2 documents : 9 800 acceptés = crédit débité ≤ 10 000	Tenue
Tenue du pic d'affluence	15 819 visiteurs, 0 sans réponse, 0 % d'échec, 300 rédacteurs servis	Tenue
Annonce rapide de l'épuisement	410 en 10 ms (p95) sans DynamoDB, arrêt du client au 410	Tenue

Reste à faire avant la soutenance : un second passage du test de charge avec le générateur corrigé (courbe de retard côté serveur).